

Logimart ERP

A B2B surface & domestic-air cargo logistics platform — booking, mile-by-mile tracking, a zone rate engine, GST billing, and a customer portal. This document is the complete technical & business handover for an incoming developer.

LEGAL ENTITY

Excelex Express Logistics LLP

BRAND / PRODUCT

Logimart

LIVE AT

erp.logimart.co.in

PREPARED

30 Aug 2026

01

What it is

A single platform that runs the whole logistics workflow for a cargo carrier — from a customer's booking to the delivered POD to the GST invoice — with pricing, carrier hand-off, and reporting built in.

Logimart replaces spreadsheets and disconnected tools with one system covering four operational domains:

Operate

Book shipments (single, bulk, walk-in, or self-service portal), assign carriers, and track each consignment through First / Mid / Last mile via scan events, ending in a mandatory proof-of-delivery image.

Bill

Price every shipment from customer-specific, zone-based rate cards (weight or per-box), raise consolidated GST invoices & e-invoices, manage debit/credit notes, and reconcile vendor cost bills into P&L.

Report

MIS, SLA (on-time vs delay), NDR, statements and AR — viewable on screen, exportable to Excel, and auto-emailed each morning. Customer-facing invoice & delivery emails go out automatically.

Control

Role-based access with per-user, per-feature View / Edit / Delete rights; masters for pincodes, zones, products, fuel, and vendors; a full audit log; and a Flutter rider app for field pickup/delivery.

Business requirements (BRD)

The functional scope, grouped by domain. Every item below is built and live in the current codebase (41 backend modules, 49 data models).

2.1 Customers & pricing

Customer master

B2B accounts with GSTIN/PAN, credit terms & limits, wallet/credit/cash/card account types, sales & CS owners, account grouping (head-office → child accounts), and a configurable auto-generated customer-code series.

Rate cards (zone engine)

Per customer × network (SELF or a vendor) × product, with pincode-driven origin/dest zones, weight slabs, fuel surcharge (flat % or diesel-indexed), FOV, ODA/EDL, city-special rates, and min-freight rules. An isolated *per-box* engine prices box-count consignments separately without touching the weight engine.

Rate tools

Bulk rate upload (Excel matrix, many customers/vendors/products in one workbook), bulk %/₹ rate changes, copy-cards-between-customers, and export in the re-uploadable template layout.

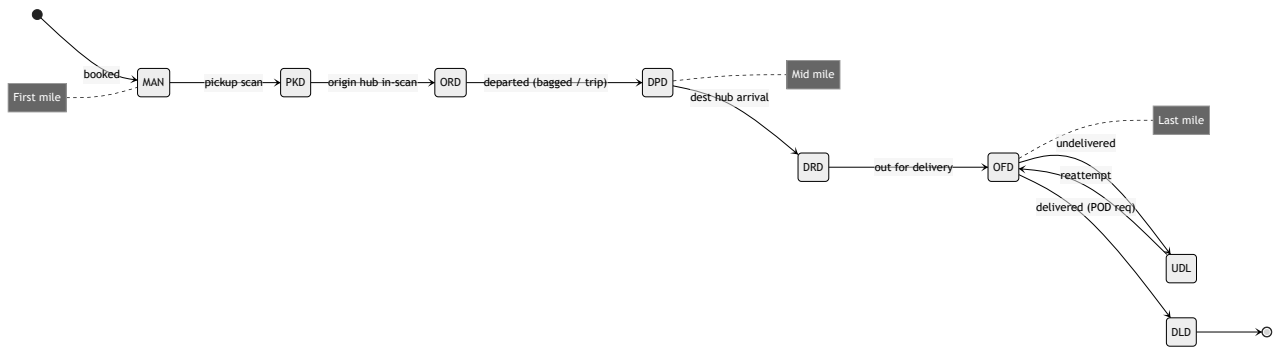
2.2 Booking & operations

Shipment booking

AWB creation with consignor/consignee, pieces & volumetric weight, product/service, e-way bill & compliance fields, carrier assignment (with a cheapest-first rate comparison), and saved pickup warehouses. Single, bulk CSV, walk-in counter, and customer-portal booking.

Mile lifecycle (scan-driven)

Status codes MAN → PKD → ORD → DPD → DRD → OFD → DLD / UDL across First-, Mid- (bagging, trips) and Last-mile, each advanced by a scan. Delivery (DLD) requires a POD image. Pickups (single & bulk upload) assigned to riders.



Each transition is a scan event appended to ScanLog; the shipment's `statusCode` only advances along a permitted edge (a super-admin can override). DLD is terminal and blocked without a POD image.

2.3 Billing & finance

Invoicing

Consolidated invoices (single / selected / all customers), lock-to-issue, GST e-invoice IRN generation, add-on charges, reattempt/demurrage & claim credit notes with print-ready PDFs, and a customer statement & AR aging.

Vendor cost & P&L

Vendor rate cards (cost side), vendor bill upload & reconciliation, and per-AWB / customer / vendor profit-and-loss with +/0/- filters.

2.4 Reports, notifications & portal

Reports

MIS, Delivery status, Manifest, Pickup, DRS, Scan family, Customer/Product/Destination summaries, Billing, Invoice, Receivables, Ledger aging, and SLA (on-time/delay summary + per-AWB). All Excel-exportable.

Notifications (email / WhatsApp)

Auto-scheduled morning NDR+MIS digest (08:00 IST), invoice-issued email with portal link, delivery-confirmation email with tracking link. Email over Brevo HTTPS API (or SMTP); WhatsApp via Meta Cloud API.

Customer portal

Simplified self-service booking (ops/pricing fields hidden), multi-AWB tracking, rate-check estimator, appointment scheduling and per-AWB remarks.

2.5 Administration

RBAC & audit

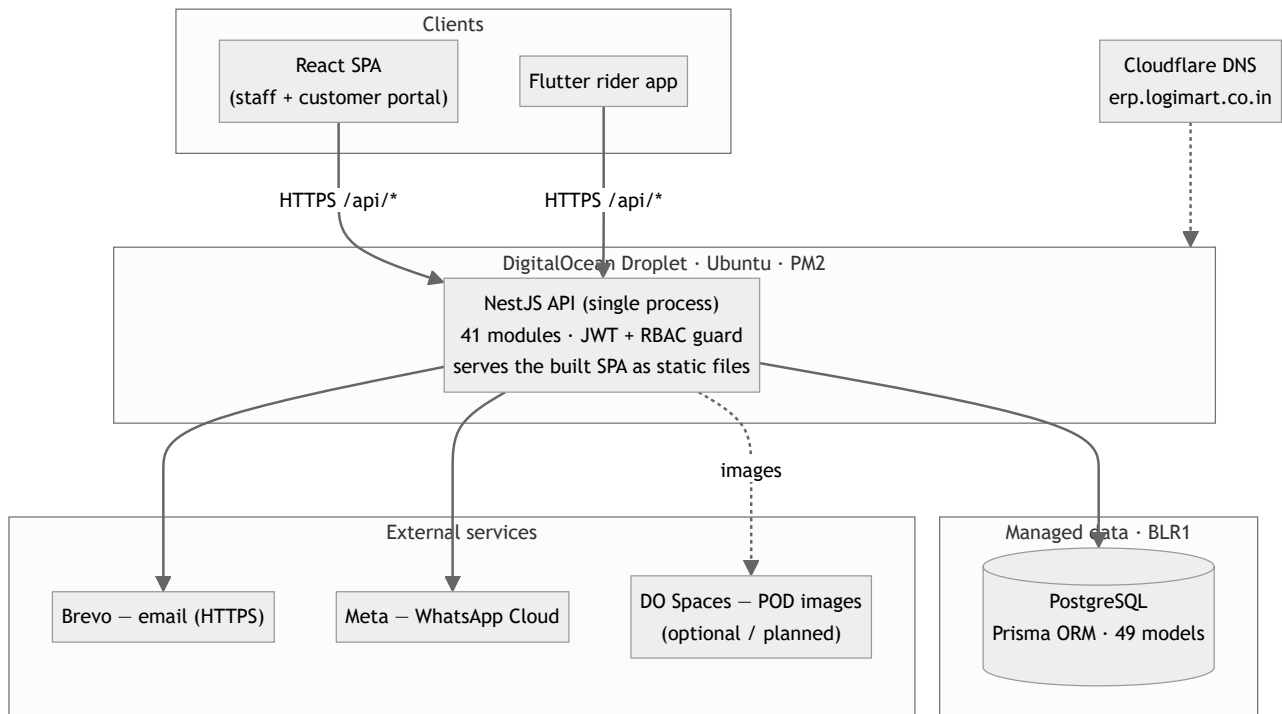
Roles (Customer, Finance, Ops, Sales, Admin, Super Admin) plus per-user, per-feature **View / Edit / Delete** grants. Full action & login audit log. Super-admin bulk delete and start-from-scratch wipe.

Masters & mobile

Pincodes & zones, products, fuel mechanisms, hubs, vendors & contacts, per-vendor serviceability/EDL. Flutter rider app for field pickup & delivery scans.

System architecture

A TypeScript monorepo with three apps. The NestJS API is the single backend; it also serves the compiled React SPA as static files, so one process serves the whole web experience. A Flutter app is a separate client hitting the same API.



Backend — modular monolith

NestJS with 41 feature modules (auth, shipments, billing, lifecycle, ratecards, reports, notifications, users...). Prisma is the data layer over PostgreSQL. A global RolesGuard verifies the JWT and enforces role + feature grants on every route. Scheduled jobs run in-process via @nestjs/schedule.

Frontend — SPA served by the API

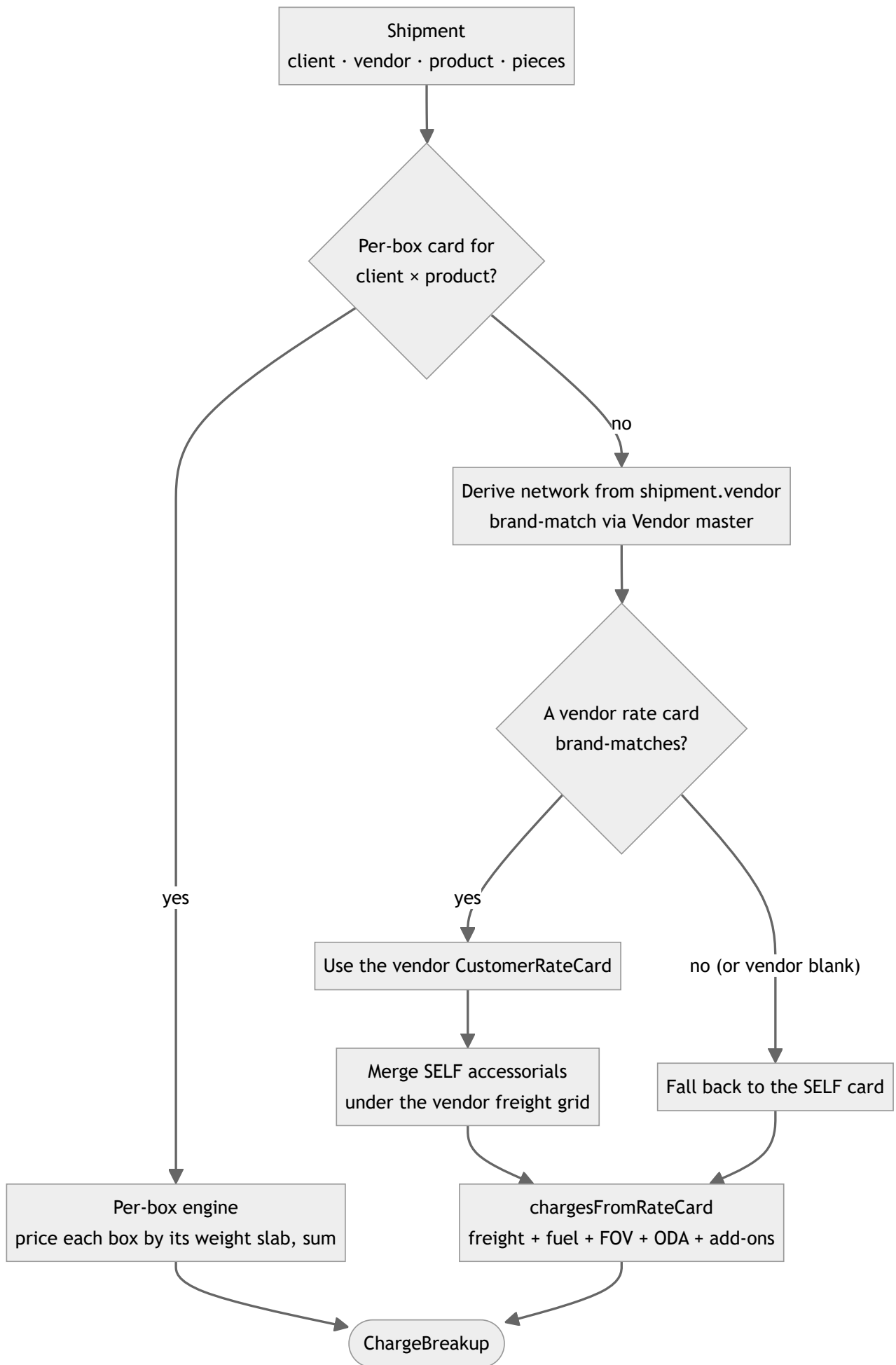
React + Vite single-page app. Built to static assets and served by the same NestJS process (ServeStaticModule, SPA fallback), with /api/* excluded so it reaches controllers. One deploy, one origin — no separate web server needed.

Key design choice. The rate engine is split: a **weight engine** (zone × weight slabs) and an **isolated per-box engine** that intercepts before it — added deliberately so box-count pricing never risks the core

weight logic. Any developer touching pricing should preserve that isolation.

3.1 How a shipment is priced

This is the single most important mechanism to understand before touching billing. Pricing keys off `shipment.vendor` — not the carrier AWB — and falls back to the SELF card when no vendor card matches (the cause of a real mis-pricing incident: a hand-off recorded the carrier AWB but left `vendor` blank, so it billed SELF rates).



The per-box engine short-circuits the weight engine entirely when a per-box card exists. Within the weight engine, vendor cards win over SELF; SELF-card accessorials still merge *under* a matched vendor card so add-ons defined once apply to every carrier.

Technology stack

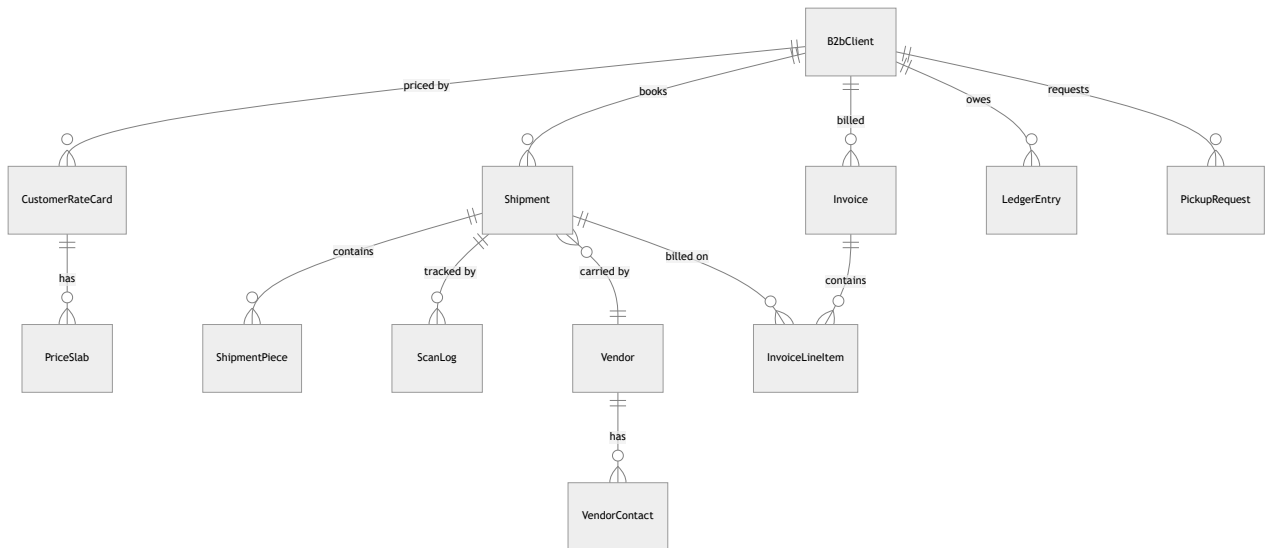
Mainstream, well-supported, TypeScript end-to-end on the server and web. Nothing exotic — an incoming developer productive in Node/React can onboard quickly.

LAYER	TECHNOLOGY	VERSION	PURPOSE
API framework	NestJS	10.4	Modular backend, DI, guards, controllers
Language	TypeScript	5.6	Server & web, fully typed
ORM	Prisma	5.20	Schema, migrations (db push), typed queries
Database	PostgreSQL	16/18	Primary datastore (49 models)
Auth	@nestjs/jwt + bcrypt	10.2	JWT bearer tokens, hashed passwords, RBAC
Scheduling	@nestjs/schedule	4.1	Cron jobs (morning digest)
Email	Brevo API / nodemailer	9.0	Transactional email (HTTPS or SMTP)
Excel I/O	SheetJS (xlsx)	0.18	Rate/report import & export
Validation	class-validator	0.14	DTO validation on controllers
Web framework	React + react-router	18.3 / 6.26	SPA, client routing
Web bundler	Vite	5.4	Dev server & production build
Mobile	Flutter (Dart)	—	Rider pickup/delivery app
Process mgr	PM2	—	Keeps the API alive, restarts, logs
TLS	Let's Encrypt (certbot)	—	Free auto-renewing HTTPS
DNS	Cloudflare	—	erp.logimart.co.in

Data model (core entities)

49 Prisma models. The dozen below are the spine an incoming developer should learn first; the rest hang off them (charges, notes, scan logs, masters, audit, etc.).

MODEL	ROLE
B2bClient	Billing customer — terms, credit, account type, grouping, sales/CS owners
Shipment	The AWB — parties, pieces, product, vendor, status/statusCode, POD, e-way bill
ShipmentPiece	Per-box dead/volumetric weight and status
CustomerRateCard + PriceSlab	Sell-side pricing per customer × network × product; zone×weight slabs
PerBoxRateCard + PerBoxSlab	Isolated box-count pricing engine
Invoice + InvoiceLineItem	GST invoices, e-invoice IRN, line items from shipments & add-ons
LedgerEntry / DebitCreditNote	Customer AR ledger and D/C notes
Vendor + VendorContact	Carriers, service modes, branch contacts
ScanLog	Append-only milestone history driving the lifecycle
PickupRequest	Pickup jobs (single & bulk) assigned to riders
Pincode / MasterEntry	Zones, serviceability, and all configurable masters (products, fuel, config)
User	Staff/rider/customer logins — role + per-feature grants



The core relationships. **B2bClient** is the hub of the sell side (bookings, rate cards, invoices, ledger); **Shipment** is the hub of operations (pieces, scans, carrier, invoice lines). Masters (pincodes, products, fuel, config) and audit sit outside this core.

Deployment & infrastructure

Fully isolated on the customer's own DigitalOcean account (kept separate from any other apps) — India-resident data (BLR1). One droplet runs the API; a managed database holds the data.

COMPONENT	DETAIL
Compute	DigitalOcean droplet (1 vCPU / 2 GB RAM / ~50 GB SSD), Ubuntu, PM2 process <code>logimart</code>
Database	DigitalOcean Managed PostgreSQL, BLR1 region
Domain / DNS	<code>exp.logimart.co.in</code> via Cloudflare → droplet IP
TLS	Let's Encrypt (certbot), auto-renew
Email	Brevo transactional API (free tier 300/day); SMTP fallback to <code>mail.logimart.co.in</code>
Image storage	Currently inline in Postgres; DO Spaces (BLR1) planned for POD/label images
Config	All secrets in <code>apps/api/.env</code> on the droplet (git-ignored) — DB URL, JWT secret, Brevo/SMTP, company GSTIN

Deploy flow

Work on branch `feat/logimart-uat` → force main → push. On the droplet: `git fetch && git reset --hard origin/main`, then `npm install && npm run build` in `apps/api` (Prisma db push applies additive schema; Vite builds the SPA), then `pm2 restart logimart --update-env`.

Known rough edge to fix. Auto-deploy conflicts on a droplet-modified `package-lock.json` — the deploy script should use `git reset --hard` (or `npm ci`) so pulls never block. Worth hardening early.

Cost to build

What this platform represents in engineering effort — a from-scratch rebuild estimate. It is a broad logistics ERP: 41 backend modules, 49 data models, ~40 web screens, a rate engine, GST billing, a customer portal, and a mobile app.

WORK PACKAGE	EFFORT	NOTES
Foundation — auth, RBAC, schema, project setup	3–4 wk	JWT, roles + feature grants, Prisma models
Rate engine — zones, weight + per-box, fuel/FOV/ODA	8–10 wk	The hardest part; two isolated engines
Booking — single / bulk / walk-in / portal + carrier	5–6 wk	Volumetric weight, compliance fields
Mile lifecycle — First/Mid/Last, bagging, trips, POD	6–8 wk	Scan-driven state machine
Billing — invoices, GST e-invoice, notes, ledger, vendor P&L	7–9 wk	Money path; e-invoice IRP integration
Reports — MIS / SLA / NDR / statements + Excel export	3–4 wk	Many report types
Customer portal + notifications (email/WhatsApp)	5–6 wk	Self-service booking, tracking, alerts
Flutter rider app	4–5 wk	Field pickup/delivery scans
Masters, pincodes/zones, admin, audit, integrations	5–6 wk	Brevo, BlueDart, config masters
Testing, deployment, hardening, iteration	5–7 wk	Build gates, fixes, UAT feedback

Total effort ≈ 15–18 person-months (≈ 12–16 dev-months of build + PM/QA overhead). Deliverable as one strong full-stack lead over ~12–14 months, or a small team (2–3 devs) over ~6–8 months.

LEAN / SOLO

₹22–28 L

≈ \$26k–33k · one-time

- One senior full-stack dev, ~12–14 months
- India rate ≈ ₹1.8–2.0 L / dev-month
- Minimal PM/QA; founder-driven UAT

most realistic

SMALL TEAM

₹30–40 L

≈ \$36k–48k · one-time

- 2–3 devs + part-time Flutter, QA & PM
- ~6–8 months to this scope
- India blended studio rates

WESTERN AGENCY

\$120k–250k

≈ ₹1.0–2.1 Cr · one-time

- Same scope at US/EU agency rates
- 3–5× the India cost
- Shown for benchmark only

Lowest realistic cost to build this from scratch

₹22–28 L · ≈ \$26k–33k, one-time

Indicative at Indian market rates (senior full-stack ≈ ₹1.8–2.0 L/dev-month; USD ≈ ₹84). Excludes third-party licence/gateway fees and the running cost below. A build reusing an existing ERP blueprint (as this one did) lands at the lower end; a pure greenfield build trends higher.

Cost to run (monthly)

Separate from the one-time build — infrastructure is deliberately lean. Three tiers, rock-bottom to comfortable.

ROCK-BOTTOM

\$12/mo

≈ ₹1,010 / mo

- 1 droplet (2 GB) — runs API + a self-hosted Postgres on the same box
- Free: DNS, TLS, Brevo email
- No managed backups / HA — acceptable for pilot, not ideal for live billing data

recommended minimum

LEAN PRODUCTION

\$27/mo

≈ ₹2,270 / mo

- Droplet (2 GB) — \$12
- Managed PostgreSQL (1 GB) — \$15 — automated backups, PITR
- Free: DNS, TLS, Brevo (300 emails/day)

COMFORTABLE

\$44/mo

≈ ₹3,700 / mo

- Droplet 4 GB — \$24
- Managed PostgreSQL 1 GB — \$15
- DO Spaces (POD images, 250 GB) — \$5

Lowest realistic to run the app in production

\$27/mo · ≈ ₹2,270

LINE ITEM	LOWEST	NOTES
DO Droplet (1 vCPU, 2 GB)	\$12 / mo	Runs the single API process (serves web too)
DO Managed PostgreSQL (1 GB)	\$15 / mo	Backups + point-in-time recovery; can be self-hosted on the droplet to save this
Domain	~\$12 / yr	≈ ₹1,000/yr; currently partner-provided
Cloudflare DNS	\$0	Free plan is sufficient
TLS / HTTPS	\$0	Let's Encrypt
Email (Brevo)	\$0	Free 300/day; ~\$9/mo if volume grows past that
WhatsApp (optional)	usage-based	Only if enabled; per-conversation pricing
DO Spaces (optional)	\$5 / mo	When POD images move off the database

Prices are DigitalOcean list rates; INR at ≈ ₹84/USD. Excludes developer time, the Anthropic/AI or BlueDart integrations (not required to run), and any GST e-invoice / IRP gateway fees.

Developer onboarding checklist

What a new developer needs to get running and take ownership.

1. **Repo access** — the monorepo (apps/api, apps/web, apps/mobile) and the developer onboarding guide already in the repo.
2. **Local setup** — Node 20, a local Postgres, copy `deploy/env.production.template` → `apps/api/.env`, `npm install`, `npx prisma generate && npx prisma db push`, `npm run start:dev` (API) and `npm run dev` (web).
3. **Build gate** — always verify web with `npm run build` (tsc + Vite); a web type error silently aborts the deploy.
4. **Credentials to obtain from the owner** — DigitalOcean account, droplet SSH, managed DB connection string, JWT secret, Brevo API key, mail-server mailbox, Cloudflare DNS, company GSTIN.
5. **Deploy access** — droplet IP + SSH; understand the `git reset --hard` → `build` → `pm2 restart` flow; deploy at the low-traffic window.
6. **First hardening tasks** — fix the auto-deploy lock-file conflict; move POD images to DO Spaces (BLR1); finish the email sender verification (Brevo domain auth or DO SMTP unblock).

Logimart ERP — technical & business handover · Excelex Express Logistics LLP · prepared 30 Aug 2026. Figures are indicative DigitalOcean list pricing; confirm current rates at deploy time.